Binary Trees — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

244-246. Introduction to Trees & Representation

Understanding the Problem NEW

A Tree is a hierarchical structure (unlike the linear structures — arrays, linked lists, stacks, queues — covered so far): it has a single 'root' node at the top, and each node can have multiple 'children' nodes below it, but no cycles (you can never follow child links back to an ancestor). A Binary Tree specifically restricts every node to at most 2 children, conventionally called 'left' and 'right.'

Brute Force Approach NEW

Nothing to contrast against here — this is purely foundational vocabulary needed before tackling any tree problem.

Why We Optimize NEW

Trees are a natural fit whenever data has an inherent hierarchy (file systems, org charts, decision processes, or — as you'll see repeatedly in this chapter — anything that benefits from 'divide into left part and right part' thinking, echoing the divide-and-conquer ideas already seen in Merge Sort and Quick Sort). Representing a binary tree is simple: each node just stores its own value plus a reference to its left child and right child (null if absent) — and almost every algorithm on trees, as you'll see throughout this chapter, is naturally expressed recursively, since a tree is inherently a recursive structure (a tree's left child is itself the root of a smaller tree, and so is its right child).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Understanding tree data structure and representation.

Visual Diagram

```
Binary Tree Structure:
                    1
                  /    \
                2       3
               / \     / \
             4     5 6    7
          Terminology:
```

```
       - Root: Node 1
       - Parent: Node 1 is parent of 2,3
       - Child: Nodes 2,3 are children of 1
       - Leaf: Nodes 4,5,6,7
       - Height: 3 (levels: 1,2,3)

       Java Representation:
       class Node {
           int data;
           Node left;
           Node right;
           Node(int data) {
               this.data = data;
               this.left = null;
               this.right = null;
           }
       }
```

Java Implementation

```
class Node {
           int data;
           Node left;
           Node right;

           Node(int data) {
               this.data = data;
               this.left = null;
               this.right = null;
           }
       }

       class BinaryTree {
           Node root;

           BinaryTree() {
               root = null;
           }

           // Create sample tree
           public void createSampleTree() {
               root = new Node(1);
               root.left = new Node(2);
               root.right = new Node(3);
               root.left.left = new Node(4);
               root.left.right = new Node(5);
               root.right.left = new Node(6);
               root.right.right = new Node(7);
           }
       }
```

247. Binary Tree Traversals

Understanding the Problem NEW

There are several standard ways to visit ('traverse') every node in a binary tree, differing only in the *order* nodes are visited: Preorder (Root, then Left, then Right), Inorder (Left, then Root, then Right), Postorder (Left, then Right, then Root), and Level Order (visit level by level, top to bottom, left to right within each level).

Brute Force Approach NEW

There's no 'brute' alternative — these are simply different, equally valid definitions of 'visit order,' and the goal here is understanding when each ordering is useful (e.g. Inorder on a Binary Search Tree visits nodes in sorted order, which becomes very useful later).

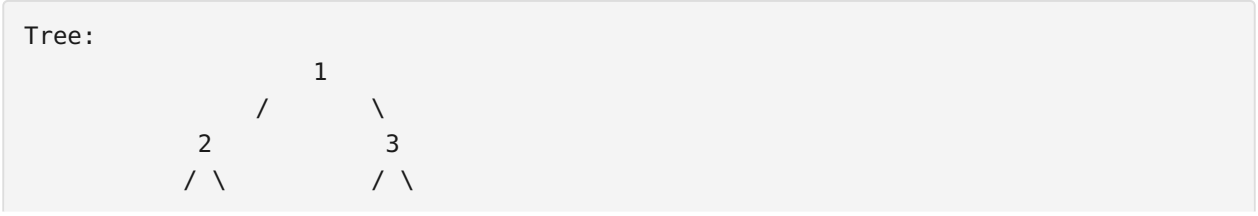
Why We Optimize NEW

The first three orderings (Pre/In/Post) share the exact same recursive shape — visit some combination of 'current node, recurse left, recurse right' — just with the current node's visit slotted into a different position relative to the two recursive calls, which is why they're grouped together and implemented almost identically (problems 248-250). Level Order is fundamentally different, requiring a queue-based breadth-first approach (problem 251) rather than simple recursion, since 'level by level' doesn't map naturally onto the tree's own recursive left/right structure — this contrast between depth-first (recursive) and breadth-first (queue-based) traversal is a distinction that becomes central again in the Graphs chapter.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Understanding different traversal orders.

Visual Diagram



```
   4    5 6    7

Preorder: 1, 2, 4, 5, 3, 6, 7 (Root, Left, Right)
Inorder: 4, 2, 5, 1, 6, 3, 7 (Left, Root, Right)
Postorder: 4, 5, 2, 6, 7, 3, 1 (Left, Right, Root)
Level Order: 1, 2, 3, 4, 5, 6, 7
```

248. Preorder Traversal

Understanding the Problem NEW

Visit every node in Root → Left → Right order — process the current node first, then fully explore its entire left subtree, then its entire right subtree.

Brute Force Approach NEW

Without recursion, you could try to simulate this manually with an explicit stack tracking 'what to visit next,' which is exactly what the iterative version (problem 252-255) does — but attempting it without any systematic bookkeeping at all (just ad-hoc pointer-chasing) quickly becomes error-prone for anything beyond simple trees.

Why We Optimize NEW

Because a tree is inherently recursive (each subtree is itself a smaller tree), the algorithm is almost a direct restatement of the definition: visit the current node, then **recursively** call the same function on the left child, then recursively call it on the right child. Each node is visited exactly once, giving O(n) time; the recursion depth (and therefore extra space used by the call stack) is O(h), the tree's height — which is O(log n) for a balanced tree but can degrade to O(n) for a completely skewed tree (essentially a linked list in disguise), a distinction that matters again in problem 258 (balance checking).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Traverse tree in Root-Left-Right order.

Visual Diagram

```
Traversal steps:
                1
              /   \
            2       3
           / \     / \
         4     5 6    7

      Visit 1 → go left
```

```
Visit 2 → go left
Visit 4 → left done, right done, back to 2
Visit 5 → left done, right done, back to 1
Visit 3 → go left
Visit 6 → left done, right done, back to 3
Visit 7

Result: [1, 2, 4, 5, 3, 6, 7]
```

Solution Approach

Concept: Recursively visit Root, Left, Right.

Algorithm

```
Algorithm PREORDER(node):
           If node == null: Return
           print(node.data)
           PREORDER(node.left)
           PREORDER(node.right)
```

Java Implementation

```
class Solution {
           // Recursive
           public void preorder(Node root) {
               if (root == null) return;
               System.out.print(root.data + " ");
               preorder(root.left);
               preorder(root.right);
           }

            // Iterative
            public List<Integer> preorderIterative(Node root) {
                List<Integer> result = new ArrayList<>();
                if (root == null) return result;

                 Stack<Node> stack = new Stack<>();
                 stack.push(root);
                 while (!stack.isEmpty()) {

                  Node node = stack.pop();
                  result.add(node.data);
                  if (node.right != null) stack.push(node.right);
                  if (node.left != null) stack.push(node.left);
              }
              return result;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

249. Inorder Traversal

Understanding the Problem NEW

Visit every node in Left → Root → Right order — fully explore the left subtree first, then process the current node, then fully explore the right subtree.

Brute Force Approach NEW

Same as Preorder — recursion is the natural approach; a non-recursive version needs an explicit stack, covered iteratively in problem 252-255.

Why We Optimize NEW

Identical recursive shape to Preorder, just with the 'visit current node' step moved to happen *between* the two recursive calls instead of before them. This ordering has a special property worth remembering: applied to a **Binary Search Tree** (where left children are always smaller and right children always bigger), Inorder traversal visits every node in fully sorted order — a fact that will become very useful once BST-specific problems appear in the next chapter. Same O(n) time, O(h) space as Preorder.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Traverse tree in Left-Root-Right order.

Visual Diagram

```
Tree:
                    1
                  / \
              2      3
            / \    / \
          4    5 6    7

       Traversal steps:
       Go left from 1 → 2
       Go left from 2 → 4
       Visit 4
       Back to 2, Visit 2
       Go right from 2 → 5
```

```
Visit 5
Back to 1, Visit 1
Go right from 1 → 3
Go left from 3 → 6
Visit 6
Back to 3, Visit 3
Go right from 3 → 7
Visit 7

Result: [4, 2, 5, 1, 6, 3, 7]
```

Solution Approach

Concept: Recursively visit Left, Root, Right.

Algorithm

```
Algorithm INORDER(node):
           If node == null: Return
           INORDER(node.left)
           print(node.data)
           INORDER(node.right)
```

Java Implementation

```
class Solution {
           // Recursive
           public void inorder(Node root) {
               if (root == null) return;
               inorder(root.left);
               System.out.print(root.data + " ");
               inorder(root.right);
           }

            // Iterative
            public List<Integer> inorderIterative(Node root) {
                List<Integer> result = new ArrayList<>();
                Stack<Node> stack = new Stack<>();
                Node current = root;

                     while (current != null || !stack.isEmpty()) {
                         while (current != null) {

                          stack.push(current);
                          current = current.left;
                       }
                       current = stack.pop();
                       result.add(current.data);
                       current = current.right;
             }
```

```
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

250. Postorder Traversal

Understanding the Problem NEW

Visit every node in Left → Right → Root order — fully explore both subtrees before processing the current node itself.

Brute Force Approach NEW

Same recursive default as the other two; the iterative version (needing to ensure both children are processed before the parent, which is trickier than Preorder/Inorder) is covered in problem 252-255.

Why We Optimize NEW

Same recursive shape again, with 'visit current node' now happening *last*, after both recursive calls. This 'process children before the parent' order is exactly what's needed whenever a computation about a node **depends on results from its subtrees** first — which is precisely the pattern behind Height (257), Diameter (259), Balanced Check (258), and Maximum Path Sum (260) later in this chapter; recognizing 'this needs postorder-style children-first computation' is one of the most useful pattern-recognition skills for tree problems.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Traverse tree in Left-Right-Root order.

Visual Diagram

```
Tree:
                    1
                  / \
              2      3
            / \    / \
          4    5 6    7

       Result: [4, 5, 2, 6, 7, 3, 1]

       Traversal steps:
       Go left from 1 → 2
       Go left from 2 → 4
```

```
Visit 4
Back to 2, go right → 5
Visit 5
Back to 2, Visit 2
Back to 1, go right → 3
Go left from 3 → 6
Visit 6
Back to 3, go right → 7
Visit 7
Back to 3, Visit 3
Back to 1, Visit 1
```

Solution Approach

Concept: Recursively visit Left, Right, Root.

Algorithm

```
Algorithm POSTORDER(node):
           If node == null: Return
           POSTORDER(node.left)
           POSTORDER(node.right)
           print(node.data)
```

Java Implementation

```
class Solution {
           // Recursive
           public void postorder(Node root) {
               if (root == null) return;
               postorder(root.left);
               postorder(root.right);
               System.out.print(root.data + " ");
           }

            // Iterative (using two stacks)
            public List<Integer> postorderIterative(Node root) {
                List<Integer> result = new ArrayList<>();
                if (root == null) return result;

                     Stack<Node> stack1 = new Stack<>();
                     Stack<Node> stack2 = new Stack<>();
                     stack1.push(root);

             while (!stack1.isEmpty()) {
                 Node node = stack1.pop();
                 stack2.push(node);
                 if (node.left != null) stack1.push(node.left);
                 if (node.right != null) stack1.push(node.right);
             }
```

```
            while (!stack2.isEmpty()) {
                result.add(stack2.pop().data);
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

251. Level Order Traversal

Understanding the Problem NEW

Visit nodes level by level, from top to bottom, and left to right within each level — e.g. for a tree with root 1 and children 2,3, level order visits 1, then 2, then 3.

Brute Force Approach NEW

Trying to force this into a simple recursive Pre/In/Post-style function doesn't work naturally, since those visit depth-first (diving all the way down one branch before moving to a sibling) — level order fundamentally needs to process nodes breadth-first, one whole layer at a time.

Why We Optimize NEW

Use a **queue**: start by enqueuing the root, then repeatedly dequeue a node, process it, and enqueue its children (left then right) — since a queue is FIFO, this naturally processes nodes in the order they were discovered, which corresponds exactly to level-by-level order. This is the tree-specific instance of **Breadth-First Search (BFS)**, a technique that reappears as a core algorithm in the Graphs chapter — trees being a special case of graphs with no cycles. O(n) time and O(width of tree) space for the queue, which can be up to O(n/2) for a wide tree.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Traverse tree level by level.

Visual Diagram

```
Tree:
                    1
                /  \
            2       3
            / \    / \
          4    5 6    7

      Result: [[1], [2, 3], [4, 5, 6, 7]]

      Queue process:
      Step 1: queue=[1]
```

```
       Step 2: pop 1 → queue=[2,3], level=[1]
       Step 3: pop 2 → queue=[3,4,5], level=[2]
       Step 4: pop 3 → queue=[4,5,6,7], level=[3]
       Step 5: pop 4,5,6,7 → queue=[], level=[4,5,6,7]
```

Solution Approach

Concept: Use queue for BFS.

Algorithm

```
Algorithm LEVEL_ORDER(root):
           If root == null: Return []
           queue = Queue()
           queue.add(root)
           result = []

            While queue not empty:
                levelSize = queue.size()
                level = []
                For i = 0 to levelSize-1:
                    node = queue.poll()
                    level.add(node.data)
                    If node.left != null: queue.add(node.left)
                    If node.right != null: queue.add(node.right)
                result.add(level)

            Return result
```

Java Implementation

```
class Solution {
           public List<List<Integer>> levelOrder(Node root) {
               List<List<Integer>> result = new ArrayList<>();
               if (root == null) return result;

                     Queue<Node> queue = new LinkedList<>();
                     queue.add(root);

                     while (!queue.isEmpty()) {
                         int levelSize = queue.size();
                         List<Integer> level = new ArrayList<>();

                         for (int i = 0; i < levelSize; i++) {

                      Node node = queue.poll();
                      level.add(node.data);
                      if (node.left != null) queue.add(node.left);
                      if (node.right != null) queue.add(node.right);
                   }
```

```
                result.add(level);
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

252-255. Iterative Traversals

Understanding the Problem NEW

Reimplement Preorder, Inorder, and Postorder traversal — but iteratively, using an explicit stack, instead of relying on recursion (which uses the call stack implicitly).

Brute Force Approach NEW

You could avoid a stack entirely and try to track position with manual pointer bookkeeping, but without a stack-like structure to remember 'where to come back to,' correctly handling arbitrary tree shapes without recursion becomes very difficult to get right.

Why We Optimize NEW

Since recursion itself is implemented (under the hood) using a call stack, using an **explicit stack** directly is a natural way to simulate the same recursive behavior without actual function calls. Preorder is the simplest to convert (push right child then left child, so left gets processed first — mirroring the natural recursive order); Inorder needs a bit more care, pushing all left children first before processing, then moving right; Postorder is the trickiest (since 'visit last' doesn't map naturally onto a simple single-stack pop order), commonly solved with either two stacks (reversing a modified preorder) or one stack with additional state tracking. This exercise builds comfort with the general principle that **any recursive algorithm can be rewritten iteratively using an explicit stack** — useful when recursion depth might risk a stack overflow on very deep/skewed trees.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

All iterative traversal implementations.

Java Implementation

```
class Solution {
           // Iterative Preorder: Root-Left-Right
           public List<Integer> preorderIterative(Node root) {
               List<Integer> result = new ArrayList<>();
```

```
    if (root == null) return result;

    Stack<Node> stack = new Stack<>();
    stack.push(root);

    while (!stack.isEmpty()) {
        Node node = stack.pop();
        result.add(node.data);
        if (node.right != null) stack.push(node.right);
        if (node.left != null) stack.push(node.left);
    }
    return result;
}

// Iterative Inorder: Left-Root-Right
public List<Integer> inorderIterative(Node root) {
    List<Integer> result = new ArrayList<>();
    Stack<Node> stack = new Stack<>();
    Node current = root;

    while (current != null || !stack.isEmpty()) {
        while (current != null) {
            stack.push(current);
            current = current.left;
        }
        current = stack.pop();
        result.add(current.data);
        current = current.right;
    }
    return result;
}

// Iterative Postorder using 2 stacks
public List<Integer> postorderIterative(Node root) {
    List<Integer> result = new ArrayList<>();
    if (root == null) return result;

    Stack<Node> stack1 = new Stack<>();
    Stack<Node> stack2 = new Stack<>();
    stack1.push(root);

    while (!stack1.isEmpty()) {
        Node node = stack1.pop();
        stack2.push(node);
        if (node.left != null) stack1.push(node.left);
        if (node.right != null) stack1.push(node.right);
    }

    while (!stack2.isEmpty()) {
        result.add(stack2.pop().data);
    }
    return result;
}
```

```
    }



256. All Traversals in One Code
```

Java Implementation

```
class Solution {
          public void allTraversals(Node root) {
              System.out.println("Preorder: " + preorder(root));
              System.out.println("Inorder: " + inorder(root));
              System.out.println("Postorder: " + postorder(root));
              System.out.println("Level Order: " + levelOrder(root));
          }

          private List<Integer> preorder(Node root) {
              List<Integer> result = new ArrayList<>();
              preorderHelper(root, result);
              return result;
          }

          private void preorderHelper(Node node, List<Integer> result) {
              if (node == null) return;
              result.add(node.data);
              preorderHelper(node.left, result);
              preorderHelper(node.right, result);
          }

          private List<Integer> inorder(Node root) {
              List<Integer> result = new ArrayList<>();
              inorderHelper(root, result);
              return result;
          }

          private void inorderHelper(Node node, List<Integer> result) {
              if (node == null) return;
              inorderHelper(node.left, result);
              result.add(node.data);
              inorderHelper(node.right, result);
          }

          private List<Integer> postorder(Node root) {
              List<Integer> result = new ArrayList<>();
              postorderHelper(root, result);
              return result;
          }

          private void postorderHelper(Node node, List<Integer> result) {
              if (node == null) return;
              postorderHelper(node.left, result);
```

```
            postorderHelper(node.right, result);
            result.add(node.data);
        }

        private List<List<Integer>> levelOrder(Node root) {
            List<List<Integer>> result = new ArrayList<>();
            if (root == null) return result;

            Queue<Node> queue = new LinkedList<>();
            queue.add(root);

            while (!queue.isEmpty()) {
                int size = queue.size();
                List<Integer> level = new ArrayList<>();
                for (int i = 0; i < size; i++) {
                    Node node = queue.poll();
                    level.add(node.data);

                if (node.left != null) queue.add(node.left);
                if (node.right != null) queue.add(node.right);
            }
            result.add(level);
        }
        return result;
    }
}
```

13.2 Medium Problems

256. All Traversals in One Code

Understanding the Problem NEW

This isn't a new algorithm — it's simply all four traversal methods (Preorder, Inorder, Postorder, Level Order) collected together in one place for easy side-by-side reference and comparison.

Brute Force Approach NEW

N/A — this is a reference/consolidation exercise rather than a new problem to solve.

Why We Optimize NEW

Seeing all four traversals together highlights how similar Pre/In/Post really are (same recursive shape, different visit-order placement) while Level Order stands clearly apart as the one requiring a fundamentally different (queue-based, breadth-first) approach — reinforcing the pattern-recognition distinction established in problem 247.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

All four traversal methods (preorder, inorder, postorder, level order) implemented together in one class for easy reference and comparison.

Java Implementation

```
class Solution {
          public void allTraversals(Node root) {
              System.out.println("Preorder: " + preorder(root));
              System.out.println("Inorder: " + inorder(root));
              System.out.println("Postorder: " + postorder(root));
              System.out.println("Level Order: " + levelOrder(root));
          }

          private List<Integer> preorder(Node root) {
              List<Integer> result = new ArrayList<>();
              preorderHelper(root, result);
              return result;
          }
```

```
    private void preorderHelper(Node node, List<Integer> result) {
        if (node == null) return;
        result.add(node.data);
        preorderHelper(node.left, result);
        preorderHelper(node.right, result);
    }

    private List<Integer> inorder(Node root) {
        List<Integer> result = new ArrayList<>();
        inorderHelper(root, result);
        return result;
    }

    private void inorderHelper(Node node, List<Integer> result) {
        if (node == null) return;
        inorderHelper(node.left, result);
        result.add(node.data);
        inorderHelper(node.right, result);
    }

    private List<Integer> postorder(Node root) {
        List<Integer> result = new ArrayList<>();
        postorderHelper(root, result);
        return result;
    }

    private void postorderHelper(Node node, List<Integer> result) {
        if (node == null) return;
        postorderHelper(node.left, result);
        postorderHelper(node.right, result);
        result.add(node.data);
    }

    private List<List<Integer>> levelOrder(Node root) {
        List<List<Integer>> result = new ArrayList<>();
        if (root == null) return result;

        Queue<Node> queue = new LinkedList<>();
        queue.add(root);

        while (!queue.isEmpty()) {
            int size = queue.size();
            List<Integer> level = new ArrayList<>();
            for (int i = 0; i < size; i++) {
                Node node = queue.poll();
                level.add(node.data);
          if (node.left != null) queue.add(node.left);
          if (node.right != null) queue.add(node.right);
      }
      result.add(level);
    }
    return result;
```

```
    }
}
```

257. Height of a Binary Tree

Understanding the Problem NEW

Find the height of a binary tree — the number of edges (or nodes, depending on convention) on the longest path from the root down to any leaf.

Brute Force Approach NEW

You could try to enumerate every root-to-leaf path explicitly and take the longest one's length — technically correct, but generating every full path when you only need the length is more work than necessary.

Why We Optimize NEW

A node's height only depends on the heights of its *children* — specifically, `height(node) = 1 + max(height(left), height(right))`, with a height of 0 for an empty/null node. This is a natural **postorder-style** recursion (children's results are needed before the current node's result can be computed, echoing the pattern flagged in problem 250): recursively compute both subtrees' heights first, then combine them for the current node. This computes the whole tree's height in a single O(n) pass, visiting each node exactly once, rather than explicitly enumerating and comparing every root-to-leaf path.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find maximum depth (height) of binary tree.

Visual Diagram

```
Tree:
                  1        height = 3
               /     \
             2         3
           / \
         4    5

      Height = max(2, 1) + 1 = 3

      Recursive:
      height(1) = 1 + max(height(2), height(3))
```

```
       height(2) = 1 + max(height(4), height(5))
       height(4) = 1 + max(0, 0) = 1
       height(5) = 1
       height(2) = 1 + max(1, 1) = 2
       height(3) = 1
       height(1) = 1 + max(2, 1) = 3
```

Solution Approach

Concept: Recursively find max depth.

Algorithm

```
Algorithm HEIGHT(node):
           If node == null: Return 0
           leftHeight = HEIGHT(node.left)
           rightHeight = HEIGHT(node.right)
           Return 1 + max(leftHeight, rightHeight)
```

Java Implementation

```
class Solution {
           public int height(Node root) {
               if (root == null) return 0;
               return 1 + Math.max(height(root.left), height(root.right));
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

258. Check if Binary Tree is Height-Balanced

Understanding the Problem NEW

Check whether a binary tree is 'height-balanced' — meaning for *every* node in the tree, the height difference between its left and right subtrees is at most 1.

Brute Force Approach NEW

A naive approach computes the height of every node's subtrees independently by calling the Height function (problem 257) separately at each node — but since Height itself is O(n), and you'd call it at every one of the n nodes, this gives O(n²) total, redundantly recomputing heights of the same lower subtrees over and over from different starting points.

Why We Optimize NEW

Instead of computing height and checking balance as two separate passes, do them **together in a single postorder traversal**: at each node, first recursively get the left and right subtree's height *and* whether they're balanced; if either subtree is already unbalanced, immediately propagate that failure upward (no need to keep computing anything further, using a sentinel value like -1 to signal 'already found unbalanced'); otherwise, check the current node's own height difference and return the combined height. This computes both height and balance-status simultaneously in one O(n) pass, instead of the O(n²) that comes from recomputing subtree heights redundantly for every node.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if tree is height-balanced (diff ≤ 1 at every node).

Visual Diagram

```
Balanced:
                  1
               /    \
             2       3
            / \
          4     5
      Height diff = |2-1| = 1 ✓
```

```
      Unbalanced:
                1
              /
            2
          /
        3
      Height diff = |3-0| = 3 ✗

      Approach: Check balance at each node
      For node 1: leftHeight=3, rightHeight=0 → diff=3 → false
```

Solution Approach

Concept: Recursively check balance and height.

Algorithm

```
Algorithm IS_BALANCED(node):
          If node == null: Return true
          left = height(node.left)
          right = height(node.right)
          If abs(left-right) > 1: Return false
          Return IS_BALANCED(node.left) && IS_BALANCED(node.right)
```

Java Implementation

```
class Solution {
          public boolean isBalanced(Node root) {
              return checkBalance(root) != -1;
          }

           private int checkBalance(Node node) {
               if (node == null) return 0;

                int left = checkBalance(node.left);
                if (left == -1) return -1;
                int right = checkBalance(node.right);
                if (right == -1) return -1;

                if (Math.abs(left - right) > 1) return -1;
                return 1 + Math.max(left, right);
           }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

259. Diameter of Binary Tree

Understanding the Problem NEW

Find the diameter of a binary tree — the length of the longest path between *any* two nodes in the tree (this path may or may not pass through the root).

Brute Force Approach NEW

For every possible pair of nodes, compute the path length between them and track the maximum — extremely expensive ($O(n^2)$ pairs, each requiring path-length computation), and doesn't exploit any tree structure.

Why We Optimize NEW

The key insight: the longest path passing *through* any specific node is exactly `height(left subtree) + height(right subtree)` (the path dips down the left side, up through the node, down the right side) — so the overall diameter is simply the **maximum of this value across every node in the tree**. Compute this during the same postorder height-computation traversal as problem 257: at each node, after getting both subtrees' heights, check if `leftHeight + rightHeight` beats the best diameter seen so far (tracked in a shared variable), then return the height as usual for the parent's use. This piggybacks the diameter calculation onto the height calculation you're already doing, achieving $O(n)$ total instead of $O(n^2)$ — the same 'compute once, reuse for every node's local check' idea as problem 258.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find longest path between any two nodes.

Visual Diagram

```
Tree:
                    1
                  /   \
               2        3
             / \
           4   5
```

```
       Diameter = 4 (path: 4→2→1→3 or 5→2→1→3)

       For each node:
       Diameter = max(leftDiameter, rightDiameter, leftHeight + rightHeight)
```

Solution Approach

Concept: Diameter = max path through node or in subtrees.

Algorithm

```
Algorithm DIAMETER(node):
           If node == null: Return {height:0, diameter:0}
           left = DIAMETER(node.left)
           right = DIAMETER(node.right)

            height = 1 + max(left.height, right.height)
            diameter = max(left.diameter, right.diameter, left.height +
right.height)
            Return {height, diameter}
```

Java Implementation

```
class Solution {
           private int diameter = 0;

            public int diameterOfBinaryTree(Node root) {
                height(root);
                return diameter;
            }

            private int height(Node node) {
                if (node == null) return 0;
                int left = height(node.left);
                int right = height(node.right);
                diameter = Math.max(diameter, left + right);
                return 1 + Math.max(left, right);
            }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

260. Maximum Path Sum

Understanding the Problem NEW

Find the maximum path sum in a binary tree — the largest possible sum along any path between two nodes (again, not necessarily through the root, and nodes can have negative values).

Brute Force Approach NEW

Check every possible pair of nodes and compute the path sum between them — $O(n^2)$ or worse, and doesn't exploit tree structure.

Why We Optimize NEW

Similar to Diameter (259), but now summing values instead of counting edges, with an added twist for negative values: at each node, recursively get the **maximum single-direction path sum contribution** from the left and right subtrees (clamped to 0 if negative, since including a negative-sum branch would only hurt — you'd rather not extend the path into it at all), then the best path *through* this node is `node.value + max(0, leftContribution) + max(0, rightContribution)`, checked against a running global maximum; but critically, the value *returned* to the parent (for use in path sums going further up the tree) can only include **one** side (left or right, not both, since a path can't branch in two directions once it leaves a node upward) — this distinction between 'best path through this node' (both sides allowed) and 'best contribution passed up' (only one side allowed) is the key subtlety, computed together in one O(n) postorder pass.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find maximum path sum between any two nodes.

Visual Diagram

```
Tree:
              -10
              / \
           9          20
                 /        \
```

```
                15         7

      Maximum path sum = 42 (15→20→7)

      For each node:
      maxSum = max(maxSum, leftMax + rightMax + node.val)
      return max(node.val + max(leftMax, rightMax), node.val)
```

Solution Approach

Concept: Track max path sum through each node.

Algorithm

```
Algorithm MAX_PATH_SUM(node):
          maxSum = -INF
          maxGain(node):
              If node == null: Return 0
              left = max(0, maxGain(node.left))
              right = max(0, maxGain(node.right))
              maxSum = max(maxSum, left + right + node.val)
              Return node.val + max(left, right)
```

Java Implementation

```
class Solution {
          private int maxSum = Integer.MIN_VALUE;

          public int maxPathSum(Node root) {
              maxGain(root);
              return maxSum;
          }

          private int maxGain(Node node) {
              if (node == null) return 0;
              int left = Math.max(0, maxGain(node.left));
              int right = Math.max(0, maxGain(node.right));
              maxSum = Math.max(maxSum, left + right + node.data);
              return node.data + Math.max(left, right);
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

261. Check if Two Trees are Identical

Understanding the Problem NEW

Check whether two binary trees are structurally identical — same shape, and the same value at every corresponding position.

Brute Force Approach NEW

You could flatten both trees into some linear representation (like a traversal sequence) and compare those — but a naive traversal-based comparison alone isn't fully reliable, since different tree shapes can sometimes produce the same traversal sequence (e.g. a single traversal order doesn't always uniquely determine a tree's shape), so this needs care to be fully correct.

Why We Optimize NEW

Direct recursive comparison is both simpler and unambiguous: two trees are identical if their root values match *and* their left subtrees are recursively identical *and* their right subtrees are recursively identical (with two null nodes trivially being 'identical,' and a null compared against a non-null being an immediate mismatch). This compares both trees simultaneously in a single O(min(n,m)) pass (stopping early the moment any mismatch is found), directly checking structure and values together rather than relying on a potentially ambiguous flattened representation.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if two binary trees are identical.

Visual Diagram

```
Tree 1:        Tree 2:
          1             1
         / \           / \
        2   3         2   3

       Identical: Both have same structure and values

       Tree 1:      Tree 2:
```

```
          1           1
         / \         / \
        2     3     2    4
       Not identical (values differ at right child)
```

Solution Approach

Concept: Recursively compare both trees.

Algorithm

```
Algorithm IS_IDENTICAL(t1, t2):
           If t1 == null and t2 == null: Return true
           If t1 == null or t2 == null: Return false
           If t1.data != t2.data: Return false
           Return IS_IDENTICAL(t1.left, t2.left) &&
                  IS_IDENTICAL(t1.right, t2.right)
```

Java Implementation

```
class Solution {
           public boolean isIdentical(Node p, Node q) {
               if (p == null && q == null) return true;
               if (p == null || q == null) return false;
               if (p.data != q.data) return false;
               return isIdentical(p.left, q.left) && isIdentical(p.right, q.right);
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

262. Zig Zag Traversal

Understanding the Problem NEW

Traverse the tree level by level (like Level Order, problem 251), but alternate the direction of each level — left-to-right, then right-to-left, then left-to-right again, and so on.

Brute Force Approach NEW

You could run ordinary level order traversal and then reverse every other level's results afterward — this works but does the reversal as a separate post-processing pass rather than integrating it into the traversal itself.

Why We Optimize NEW

Use the same BFS/queue approach as Level Order, but track a simple **direction flag** that toggles after every level; when building the result list for a level, either append values normally or insert them at the front (or just reverse that level's collected list before adding it to the final result), based on the current direction. This integrates the zigzag behavior directly into the same O(n) BFS traversal, rather than requiring a separate reversal pass — a small but direct extension of the Level Order technique from problem 251.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Traverse tree in zigzag level order.

Visual Diagram

```
Tree:
                    1
                  / \
              2      3
            / \    / \
          4    5 6    7

       Zigzag: [[1], [3,2], [4,5,6,7]]

       Queue process with direction flag:
       Level 0: left-to-right → [1]
```

```
       Level 1: right-to-left → [3,2]
       Level 2: left-to-right → [4,5,6,7]
```

Solution Approach

Concept: Level order with direction toggle.

Algorithm

```
Algorithm ZIGZAG(root):
           If root == null: Return []
           queue = Queue()
           queue.add(root)
           result = []
           leftToRight = true

           While queue not empty:
               levelSize = queue.size()
               level = new LinkedList()

                     For i = 0 to levelSize-1:
                         node = queue.poll()
                         If leftToRight:
                             level.addLast(node.data)
                         Else:
                             level.addFirst(node.data)
                         If node.left: queue.add(node.left)
                         If node.right: queue.add(node.right)

                     result.add(level)
                     leftToRight = !leftToRight

           Return result
```

Java Implementation

```
class Solution {
           public List<List<Integer>> zigzagLevelOrder(Node root) {
               List<List<Integer>> result = new ArrayList<>();
               if (root == null) return result;

                     Queue<Node> queue = new LinkedList<>();
                     queue.add(root);
                     boolean leftToRight = true;

              while (!queue.isEmpty()) {
                  int levelSize = queue.size();
                  List<Integer> level = new ArrayList<>();

                  for (int i = 0; i < levelSize; i++) {
```

```
                    Node node = queue.poll();
                    if (leftToRight) {
                        level.add(node.data);
                    } else {
                        level.add(0, node.data);
                    }
                    if (node.left != null) queue.add(node.left);
                    if (node.right != null) queue.add(node.right);
                }
                result.add(level);
                leftToRight = !leftToRight;
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

263. Boundary Traversal

Understanding the Problem NEW

Print the 'boundary' of the tree — the left edge (top to bottom), the leaf nodes (left to right), and the right edge (bottom to top), each visited without duplication where they overlap (e.g. the root and leaf boundary nodes shouldn't be printed twice).

Brute Force Approach NEW

Trying to handle this with one single traversal and complex conditional logic to detect 'is this a boundary node' on the fly is possible but genuinely awkward to get right cleanly.

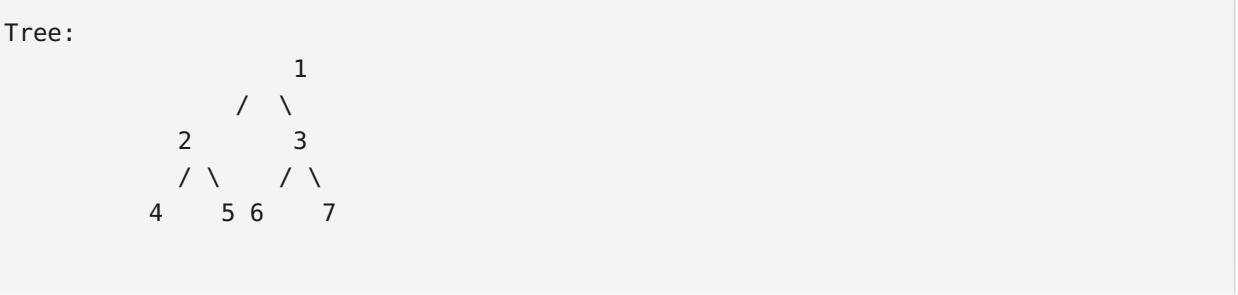
Why We Optimize NEW

Break the problem into **three clearly separate, simpler sub-traversals** and concatenate their results: (1) the left boundary, found by repeatedly going left (or right, if left is absent) from the root, excluding leaves; (2) all leaf nodes, found via any standard traversal (e.g. inorder) filtering for nodes with no children; (3) the right boundary, found similarly to the left boundary but mirrored, and printed in reverse (bottom to top) order. Decomposing a seemingly complex, unified traversal into three independent, individually simple traversals — each easy to verify correct on its own — is the real technique here, rather than trying to handle every case within one intricate traversal function.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Traverse boundary of binary tree.

Visual Diagram



```
       Boundary: [1, 2, 4, 5, 6, 7, 3]
       (Left boundary + Leaves + Right boundary reverse)

       Components:
       1. Left boundary (excluding leaves): [1, 2]
       2. Leaves: [4, 5, 6, 7]
       3. Right boundary (excluding leaves, reverse): [3]
```

Solution Approach

Concept: Three parts: left boundary, leaves, right boundary.

Algorithm

```
Algorithm BOUNDARY_TRAVERSAL(root):
           result = []
           If root == null: Return result

           If !isLeaf(root): result.add(root.data)
           addLeftBoundary(root.left, result)
           addLeaves(root, result)
           addRightBoundary(root.right, result)
           Return result
```

Java Implementation

```
class Solution {
           public List<Integer> boundaryTraversal(Node root) {
               List<Integer> result = new ArrayList<>();
               if (root == null) return result;

                    if (!isLeaf(root)) result.add(root.data);
                    addLeftBoundary(root.left, result);
                    addLeaves(root, result);
                    addRightBoundary(root.right, result);
                    return result;
           }

           private void addLeftBoundary(Node node, List<Integer> result) {
               while (node != null) {
                   if (!isLeaf(node)) result.add(node.data);
                   node = (node.left != null) ? node.left : node.right;
               }
           }

           private void addLeaves(Node node, List<Integer> result) {
               if (node == null) return;
               if (isLeaf(node)) {

                  result.add(node.data);
```

```
                return;
            }
            addLeaves(node.left, result);
            addLeaves(node.right, result);
        }

        private void addRightBoundary(Node node, List<Integer> result) {
            Stack<Integer> stack = new Stack<>();
            while (node != null) {
                if (!isLeaf(node)) stack.push(node.data);
                node = (node.right != null) ? node.right : node.left;
            }
            while (!stack.isEmpty()) {
                result.add(stack.pop());
            }
        }

        private boolean isLeaf(Node node) {
            return node.left == null && node.right == null;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

264. Vertical Order Traversal

Understanding the Problem NEW

Group nodes by their 'vertical column' — imagine dropping a vertical line through the root; nodes directly below-left get negative column numbers, below-right get positive ones, and the same column number groups nodes into the same vertical line, printed top to bottom (with a tiebreak rule for nodes at the same row and column).

Brute Force Approach NEW

You could try to compute columns without any systematic approach, but without tracking each node's column and row together, correctly grouping and ordering nodes (especially handling ties) is easy to get wrong.

Why We Optimize NEW

Do a **BFS (level order) traversal, but track each node's horizontal distance (column) and row (depth)** alongside it, storing them in a map keyed by column, with values being lists of (row, node-value) — using BFS specifically (rather than a depth-first traversal) ensures that when multiple nodes land in the same column and row, they're naturally encountered in left-to-right order, simplifying the tie-breaking rule. After the traversal, sort the map's keys (columns) and, within each column, sort by row (and value, for ties at the same row) to produce the final grouped output. This turns an initially confusing 2D-positioning problem into 'BFS while tracking extra coordinates,' directly reusing the Level Order traversal technique with additional bookkeeping.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Traverse tree by vertical columns.

Visual Diagram

```
Tree:
                  1
              /   \
           2       3
           / \     / \
```

```
          4    5 6    7

       Vertical order:
       Column -2: [4]
       Column -1: [2]
       Column 0: [1,5,6]
       Column 1: [3]
       Column 2: [7]

       HD (Horizontal Distance):
       Root: 0
       Left child: -1
       Right child: +1
```

Solution Approach

Concept: BFS with horizontal distances.

Algorithm

```
Algorithm VERTICAL_TRAVERSAL(root):
           map = new TreeMap<Integer, List<Integer>>()
           queue = Queue of (node, hd)
           queue.add((root, 0))

           While queue not empty:
               (node, hd) = queue.poll()
               map.get(hd).add(node.data)
               If node.left: queue.add((node.left, hd-1))
               If node.right: queue.add((node.right, hd+1))

           Return map values
```

Java Implementation

```
class Solution {
           public List<List<Integer>> verticalTraversal(Node root) {
               TreeMap<Integer, List<Integer>> map = new TreeMap<>();
               Queue<Pair<Node, Integer>> queue = new LinkedList<>();
               queue.add(new Pair<>(root, 0));

                    while (!queue.isEmpty()) {
                        Pair<Node, Integer> pair = queue.poll();
                        Node node = pair.getKey();
                        int hd = pair.getValue();

                        map.computeIfAbsent(hd, k -> new
ArrayList<>()).add(node.data);

                        if (node.left != null) queue.add(new Pair<>(node.left, hd -
```

```
1));
                        if (node.right != null) queue.add(new Pair<>(node.right, hd
+ 1));

                }

                return new ArrayList<>(map.values());
            }
        }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(n)

265. Top View of Binary Tree

Understanding the Problem NEW

Print the nodes visible when looking at the tree from directly above — for each vertical column (as defined in Vertical Order Traversal, 264), only the **topmost** node in that column is visible.

Brute Force Approach NEW

You could compute the full vertical order traversal (264) and then, for each column, sort by row and take the first node — correct, but requires the full grouping-and-sorting machinery of vertical order when you only need the very first node per column.

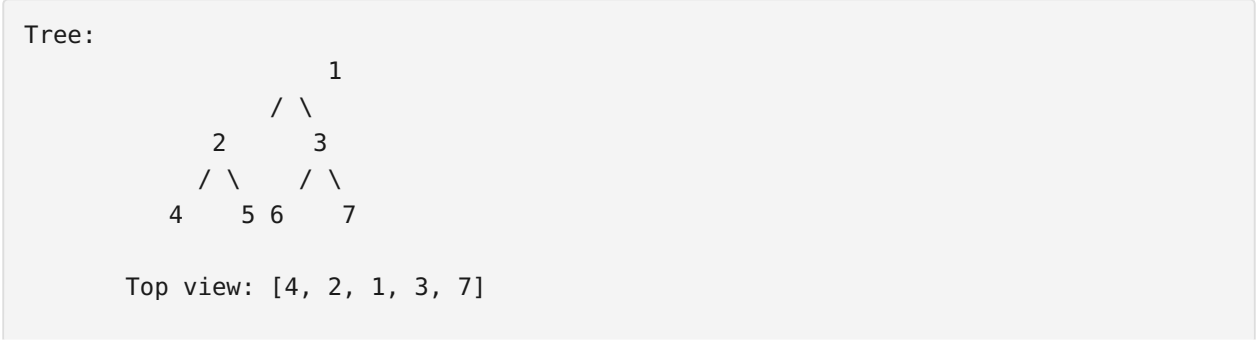
Why We Optimize NEW

Since you only need the *first* node encountered per column, and BFS naturally processes nodes in top-to-bottom order, you can simplify: do a BFS traversal tracking horizontal distance, and for each column, record a value in a map **only if that column hasn't been seen yet** (ignoring any subsequent, lower node in the same column). This achieves the same 'top view' result with a lighter-weight single pass and a simpler map (no need to store every node's row, and no post-traversal sorting by row needed), rather than fully solving the more general vertical-order problem and filtering afterward.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Print top view nodes visible from top.

Visual Diagram



```
For each HD, first node encountered in level order
HD -2: 4
HD -1: 2
HD 0: 1
HD 1: 3
HD 2: 7
```

Solution Approach

Concept: BFS with first occurrence per HD.

Algorithm

```
Algorithm TOP_VIEW(root):
           map = new TreeMap<Integer, Integer>()
           queue = Queue of (node, hd)
           queue.add((root, 0))

           While queue not empty:
               (node, hd) = queue.poll()
               If hd not in map:
                   map.put(hd, node.data)
               If node.left: queue.add((node.left, hd-1))
               If node.right: queue.add((node.right, hd+1))

           Return map values
```

Java Implementation

```
class Solution {
           public List<Integer> topView(Node root) {
               List<Integer> result = new ArrayList<>();
               if (root == null) return result;

                     Map<Integer, Integer> map = new TreeMap<>();
                     Queue<Pair<Node, Integer>> queue = new LinkedList<>();
                     queue.add(new Pair<>(root, 0));

                     while (!queue.isEmpty()) {
                         Pair<Node, Integer> pair = queue.poll();
                         Node node = pair.getKey();
                         int hd = pair.getValue();

                         if (!map.containsKey(hd)) {
                             map.put(hd, node.data);
                         }

                   if (node.left != null) queue.add(new Pair<>(node.left, hd - 1));
                   if (node.right != null) queue.add(new Pair<>(node.right, hd +
```

```
1));
            }

            return new ArrayList<>(map.values());
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

266. Bottom View of Binary Tree

Understanding the Problem NEW

The mirror image of Top View: print the nodes visible from directly below — for each vertical column, only the **bottommost** node is visible.

Brute Force Approach NEW

Same idea as Top View — you could compute the full vertical order and take the last node in each column's sorted-by-row list.

Why We Optimize NEW

Same simplified BFS approach as Top View (265), but with one key difference: instead of only recording a column's value on *first* occurrence, **always overwrite** the map's entry for that column with the current node — since BFS processes top-to-bottom, the last write for any given column will naturally end up being the bottommost node. A one-line change (always overwrite vs. only-write-if-absent) is the entire difference between Top View and Bottom View, again reinforcing that grasping the underlying BFS-with-column-tracking technique lets you solve several visually distinct-sounding problems with minimal changes.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Print bottom view nodes visible from bottom.

Visual Diagram

```
Tree:
                    1
                /   \
            2       3
            / \     / \
          4    5 6    7

       Bottom view: [4, 5, 6, 7]

       For each HD, last node encountered in level order
       HD -2: 4
       HD -1: 2 (but 5 is lower) → 5
```

```
       HD 0: 1 (but 5,6 are lower) → 5? Wait, both 5 and 6 at HD 0
       Actually bottom view: [4, 5, 6, 7]
```

Solution Approach

Concept: BFS with last occurrence per HD.

Algorithm

```
Algorithm BOTTOM_VIEW(root):
           map = new TreeMap<Integer, Integer>()
           queue = Queue of (node, hd)
           queue.add((root, 0))

           While queue not empty:
               (node, hd) = queue.poll()
               map.put(hd, node.data) // overwrite for bottom
               If node.left: queue.add((node.left, hd-1))
               If node.right: queue.add((node.right, hd+1))

           Return map values
```

Java Implementation

```
class Solution {
           public List<Integer> bottomView(Node root) {
               List<Integer> result = new ArrayList<>();
               if (root == null) return result;

                    Map<Integer, Integer> map = new TreeMap<>();
                    Queue<Pair<Node, Integer>> queue = new LinkedList<>();
                    queue.add(new Pair<>(root, 0));

                    while (!queue.isEmpty()) {
                        Pair<Node, Integer> pair = queue.poll();
                        Node node = pair.getKey();
                        int hd = pair.getValue();

                        map.put(hd, node.data); // Overwrite for bottom view

                        if (node.left != null) queue.add(new Pair<>(node.left, hd -
1));
                        if (node.right != null) queue.add(new Pair<>(node.right, hd
+ 1));
                    }

               return new ArrayList<>(map.values());
           }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

267. Right/Left View of Binary Tree

Understanding the Problem NEW

Print the nodes visible when looking at the tree from the right side (or left side) — for each *level*, only the rightmost (or leftmost) node is visible.

Brute Force Approach NEW

You could compute the full level order traversal (251) and then, for each level's list, take the last (or first) element — correct, but builds and stores every node in every level when only one per level is actually needed for the final answer.

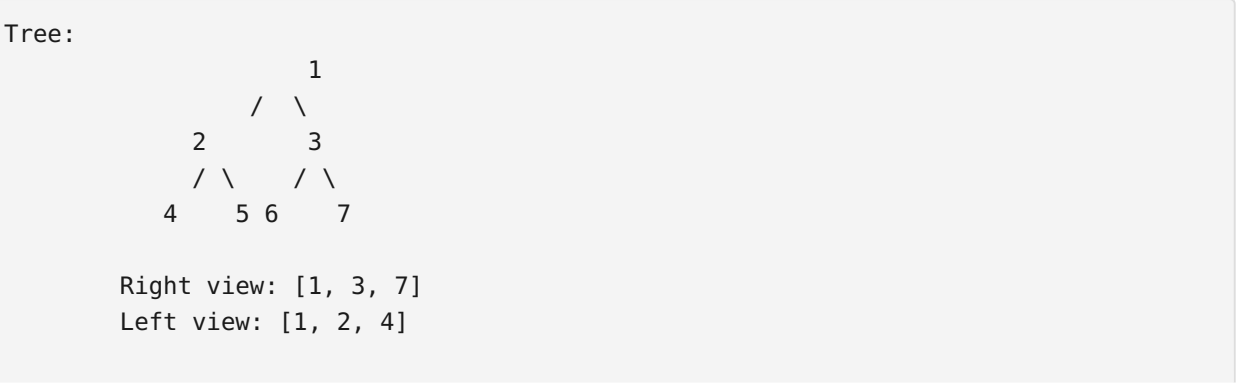
Why We Optimize NEW

Do a standard BFS, but for each level, only keep track of the **last node processed in that level** for Right View (or the *first* node for Left View) — you don't need to store the whole level's contents, just note which node was last (or first) as you dequeue through that level's nodes, adding only that one to the result. This is a lighter-weight variant of Level Order that only retains one value per level instead of the full level's list, directly building on the same BFS traversal used throughout this 'views of a tree' family of problems (Top, Bottom, Left/Right View all share the same BFS backbone, varying only in which node per grouping gets kept).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Print nodes visible from right or left side.

Visual Diagram



```
        Right view - last node at each level
        Level 0: 1
        Level 1: 3
        Level 2: 7
```

Solution Approach

Concept: Level order with first/last per level.

Algorithm

```
Algorithm RIGHT_VIEW(root):
            queue = Queue()
            queue.add(root)
            result = []

            While queue not empty:
                size = queue.size()
                For i = 0 to size-1:
                    node = queue.poll()
                    If i == size-1: result.add(node.data)
                    If node.left: queue.add(node.left)
                    If node.right: queue.add(node.right)

            Return result
```

Java Implementation

```
class Solution {
            // Right View
            public List<Integer> rightView(Node root) {
                List<Integer> result = new ArrayList<>();
                if (root == null) return result;

                     Queue<Node> queue = new LinkedList<>();
                     queue.add(root);

                     while (!queue.isEmpty()) {
                         int size = queue.size();
                         for (int i = 0; i < size; i++) {
                             Node node = queue.poll();
                             if (i == size - 1) result.add(node.data);
                             if (node.left != null) queue.add(node.left);
                             if (node.right != null) queue.add(node.right);
                         }

              }
              return result;
          }
```

```
        // Left View
        public List<Integer> leftView(Node root) {
            List<Integer> result = new ArrayList<>();
            if (root == null) return result;

            Queue<Node> queue = new LinkedList<>();
            queue.add(root);

            while (!queue.isEmpty()) {
                int size = queue.size();
                for (int i = 0; i < size; i++) {
                    Node node = queue.poll();
                    if (i == 0) result.add(node.data);
                    if (node.left != null) queue.add(node.left);
                    if (node.right != null) queue.add(node.right);
                }
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

268. Symmetric Binary Tree

Understanding the Problem NEW

Check whether a binary tree is symmetric — meaning it looks the same as its own mirror image around the vertical center line through the root.

Brute Force Approach NEW

You could try to build an explicit mirrored copy of the tree and then check if it's identical to the original (reusing the 'check identical trees' logic from problem 261) — this works, but constructing a whole extra mirrored tree just to compare it is unnecessary overhead.

Why We Optimize NEW

Rather than building a physical mirror, do a recursive comparison directly: check that the tree's **left subtree is a mirror of the right subtree** — where 'is a mirror of' means each node's value matches, its left child mirrors the other tree's right child, and its right child mirrors the other tree's left child (crossed comparisons, not same-side ones, which is the key difference from problem 261's identical-tree check). This achieves the symmetry check in a single O(n) recursive pass with no extra tree construction, by directly encoding the 'mirror' relationship into the comparison itself rather than materializing an actual mirrored copy first.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if tree is symmetric around its center.

Visual Diagram

```
Symmetric:
                1
              /   \
             2     2
            / \   / \
           3   4 4   3

      Not symmetric:
                1
              /   \
```

```
          2       2
         / \     / \
        3     4 5    3

      Mirror check:
      isMirror(left, right):
          If both null: true
          If one null: false
          If values differ: false
          Return isMirror(left.left, right.right) &&
                 isMirror(left.right, right.left)
```

Solution Approach

Concept: Mirror comparison recursively.

Algorithm

```
Algorithm IS_SYMMETRIC(root):
          If root == null: Return true
          Return isMirror(root.left, root.right)

      Algorithm isMirror(left, right):
          If left == null && right == null: Return true
          If left == null || right == null: Return false
          Return left.data == right.data &&
                 isMirror(left.left, right.right) &&
                 isMirror(left.right, right.left)
```

Java Implementation

```
class Solution {
          public boolean isSymmetric(Node root) {
              if (root == null) return true;
              return isMirror(root.left, root.right);
          }

          private boolean isMirror(Node left, Node right) {
              if (left == null && right == null) return true;
              if (left == null || right == null) return false;
              return left.data == right.data &&
                     isMirror(left.left, right.right) &&
                     isMirror(left.right, right.left);
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

269. Root to Node Path in Binary Tree

Understanding the Problem NEW

Find the path of nodes from the root down to a specific target node/value.

Brute Force Approach NEW

You could search the tree with any traversal, but without a clear way to 'remember the path so far,' correctly reconstructing the exact path taken (rather than just confirming the node exists) requires deliberate bookkeeping.

Why We Optimize NEW

Use **backtracking** during a DFS traversal: maintain a running list representing 'the path so far,' adding the current node before recursing into children; if the current node is the target, you're done (the current path list is the answer); otherwise, try recursing left, then right — if a recursive call into a child doesn't find the target, **remove that child from the path** (backtrack) before trying the other side, since that branch wasn't the correct one. This is the same 'try, recurse, undo if unsuccessful' backtracking discipline seen throughout the Recursion chapter (Word Search, Rat in a Maze, N-Queens), applied here to tree path-finding instead of grid/board exploration.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find path from root to a given node.

Visual Diagram

```
Tree:
                    1
                /  \
            2       3
            / \    / \
          4    5 6    7

       Path to 5: [1, 2, 5]

       Recursive search:
       findPath(1, 5):
```

```
         check left: findPath(2, 5)
           check left: findPath(4, 5) → false
           check right: findPath(5, 5) → add 5, return true
           add 2, return true
         add 1, return true
```

Solution Approach

Concept: Recursively search and backtrack.

Algorithm

```
Algorithm ROOT_TO_NODE(root, target):
           path = []
           If getPath(root, target, path):
               Return path
           Return []

       Algorithm getPath(node, target, path):
           If node == null: Return false
           path.add(node.data)
           If node.data == target: Return true
           If getPath(node.left, target, path) or
              getPath(node.right, target, path):
               Return true
           path.remove(path.size()-1)
           Return false
```

Java Implementation

```
class Solution {
           public List<Integer> rootToNodePath(Node root, int target) {
               List<Integer> path = new ArrayList<>();
               getPath(root, target, path);
               return path;
           }

           private boolean getPath(Node node, int target, List<Integer> path) {
               if (node == null) return false;

                    path.add(node.data);

              if (node.data == target) return true;

              if (getPath(node.left, target, path) || getPath(node.right, target,
path)) {
                   return true;
              }

              path.remove(path.size() - 1);
```

```
            return false;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

270. LCA in Binary Tree

Understanding the Problem NEW

Find the Lowest Common Ancestor (LCA) of two given nodes — the deepest node in the tree that has both target nodes as descendants (a node is considered its own descendant for this purpose).

Brute Force Approach NEW

You could find the root-to-node path for both targets separately (reusing problem 269's technique) and then compare the two paths to find where they last agree — correct, and actually a perfectly reasonable O(n) approach, but it requires storing and comparing two full paths explicitly.

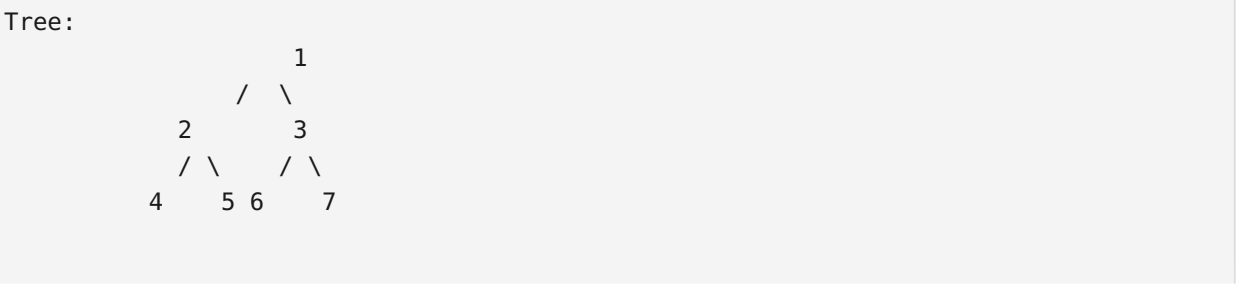
Why We Optimize NEW

A more elegant single-pass approach: recursively search both left and right subtrees for either target; if a subtree call returns a non-null result from *both* sides (meaning one target was found on the left and the other on the right), the current node *is* the LCA; if only one side returns non-null, propagate that result upward (the LCA must be further up, or that returned node itself might be the LCA if it happens to be an ancestor of the other target too). This finds the LCA in a single O(n) traversal without explicitly constructing and comparing two separate root-to-node paths, by letting the recursion itself discover where the two search paths 'merge.'

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find Lowest Common Ancestor of two nodes.

Visual Diagram



```
LCA(4, 5) = 2
LCA(4, 7) = 1
LCA(2, 5) = 2

Recursive:
findLCA(1, 4, 7):
  left = findLCA(2, 4, 7)
    left = findLCA(4, 4, 7) → returns 4
    right = findLCA(5, 4, 7) → returns null
    returns 4
  right = findLCA(3, 4, 7) → returns null
  returns 4? Wait, expected 1

Better approach:
If node is null: return null
If node is p or q: return node
left = findLCA(node.left, p, q)
right = findLCA(node.right, p, q)
If left != null && right != null: return node
Return left != null ? left : right
```

Solution Approach

Concept: Recursively find both nodes.

Algorithm

```
Algorithm LCA(node, p, q):
           If node == null: Return null
           If node == p || node == q: Return node

           left = LCA(node.left, p, q)
           right = LCA(node.right, p, q)

           If left != null && right != null: Return node
           Return left != null ? left : right
```

Java Implementation

```
class Solution {
           public Node lowestCommonAncestor(Node root, Node p, Node q) {
               if (root == null) return null;
               if (root == p || root == q) return root;

                    Node left = lowestCommonAncestor(root.left, p, q);
                    Node right = lowestCommonAncestor(root.right, p, q);

                    if (left != null && right != null) return root;

              return left != null ? left : right;
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

271. Maximum Width of Binary Tree

Understanding the Problem NEW

Find the maximum width of a binary tree — the largest number of 'slots' between the leftmost and rightmost node at any level, *including* the slots for missing (null) nodes in between (not just counting actual nodes present).

Brute Force Approach NEW

You could do ordinary level order traversal and just count how many actual nodes are present per level — but this misses the requirement to count the full span including gaps from missing nodes, e.g. a level with nodes only at the far-left and far-right position (with nothing in between) should count the full span, not just 2.

Why We Optimize NEW

During BFS, assign each node an **index** following the standard 'complete binary tree' numbering (root=0, left child = 2×parent+1, right child = 2×parent+2) — then a level's width is simply `(last index at that level) - (first index at that level) + 1`, which correctly accounts for gaps since the indices reflect true positions, not just how many nodes actually exist. A practical detail: these indices can grow very large for deep, sparse trees, so re-basing each level's indices relative to that level's first index (subtracting it off) before computing child indices prevents integer overflow — this reuses the same BFS backbone as the 'views' problems, with index-tracking added as the specific extra bookkeeping this problem needs.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find maximum width (including null nodes).

Visual Diagram

```
Tree:
                    1
                /       \
            2             3
          / \                 \
        4   5                  8
```

```
Max width = 4 (level 2: [4,5,null,8])

Using indexing:
Root: index = 1
Left: 2*index, Right: 2*index+1
Level 0: [1] → width=1
Level 1: [2,3] → width=2
Level 2: [4,5,null,8] → width=4
```

Solution Approach

Concept: BFS with indexing.

Algorithm

```
Algorithm MAX_WIDTH(root):
           queue = Queue of (node, index)
           queue.add((root, 1))
           maxWidth = 0

           While queue not empty:
               size = queue.size()
               first = queue.peek().index
               last = first

                      For i = 0 to size-1:
                          (node, index) = queue.poll()
                          last = index
                          If node.left: queue.add((node.left, 2*index))
                          If node.right: queue.add((node.right, 2*index+1))

                      maxWidth = max(maxWidth, last - first + 1)

           Return maxWidth
```

Java Implementation

```
class Solution {
           public int maxWidth(Node root) {
               if (root == null) return 0;

                      Queue<Pair<Node, Integer>> queue = new LinkedList<>();
                      queue.add(new Pair<>(root, 1));
                      int maxWidth = 0;

                      while (!queue.isEmpty()) {
                          int size = queue.size();
                          int first = queue.peek().getValue();
```

```
                int last = first;

                for (int i = 0; i < size; i++) {
                    Pair<Node, Integer> pair = queue.poll();
                    Node node = pair.getKey();
                    int index = pair.getValue();
                    last = index;

                    if (node.left != null) {
                        queue.add(new Pair<>(node.left, 2L * index));
                    }
                    if (node.right != null) {
                        queue.add(new Pair<>(node.right, 2L * index + 1));
                    }
                }
                maxWidth = Math.max(maxWidth, last - first + 1);
            }
            return maxWidth;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

272. Check for Children Sum Property

Understanding the Problem NEW

Check whether every node in the tree satisfies the 'children sum property' — meaning each non-leaf node's value equals the sum of its immediate children's values (leaf nodes are trivially fine, having no children to sum).

Brute Force Approach NEW

Check each node against its immediate children individually with a simple traversal — this is actually already efficient and there isn't a slower 'brute' alternative to meaningfully contrast against; the interesting part of this problem is usually the related task of *fixing* a tree to satisfy this property, which requires more careful top-down adjustment.

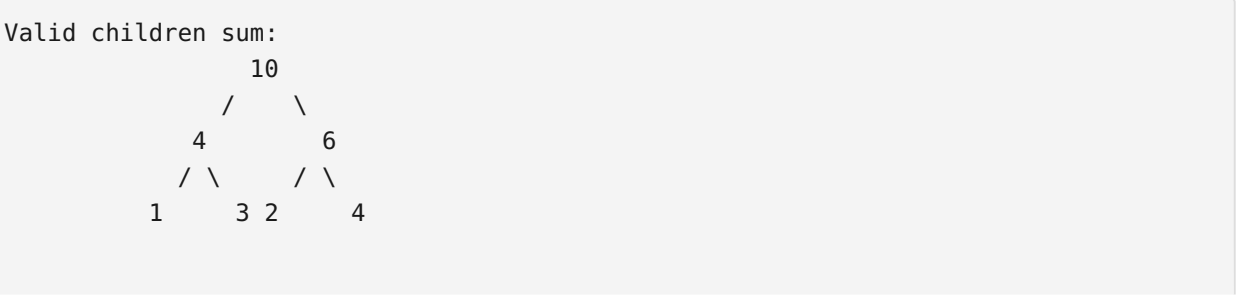
Why We Optimize NEW

A straightforward single traversal suffices to *check* the property: for each node, verify `node.value == (left child's value, or 0 if absent) + (right child's value, or 0 if absent)`, recursing into children regardless, achieving O(n) time with no special technique beyond a direct property check per node — worth noting mainly because a very similar-looking problem ('make the tree satisfy this property by adjusting values') requires a more careful top-down pass (propagating value changes downward from parent to children) rather than the simple bottom-up check shown here.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if every node equals sum of children.

Visual Diagram



```
       10 = 4+6 ✓
       4 = 1+3 ✓
       6 = 2+4 ✓

       Invalid:
                     10
                /         \
             5              4
             / \            / \
            1   3          2   4

       10 ≠ 5+4 ✗
```

Solution Approach

Concept: Recursively check children sum.

Algorithm

```
Algorithm CHILDREN_SUM(node):
           If node == null: Return true
           If node.left == null && node.right == null: Return true

            sum = (node.left != null ? node.left.data : 0) +
                  (node.right != null ? node.right.data : 0)

            Return node.data == sum &&
                   CHILDREN_SUM(node.left) &&
                   CHILDREN_SUM(node.right)
```

Java Implementation

```
class Solution {
           public boolean isChildrenSumProperty(Node root) {
               if (root == null) return true;
               if (root.left == null && root.right == null) return true;

                     int sum = 0;
                     if (root.left != null) sum += root.left.data;
                     if (root.right != null) sum += root.right.data;

                     return root.data == sum &&
                            isChildrenSumProperty(root.left) &&
                            isChildrenSumProperty(root.right);
            }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(h)

273. Nodes at Distance K in Binary Tree

Understanding the Problem NEW

Given a target node, find every node in the tree that is exactly K edges away from it — which could be nodes below the target (descendants) or nodes reached by going up through ancestors and then back down other branches.

Brute Force Approach NEW

Compute the distance from the target to every other node individually (e.g. via a fresh search from the target for each candidate node) — O(n) per node, $O(n^2)$ overall, since 'distance in a tree' isn't naturally something a simple top-down recursion computes directly for arbitrary node pairs.

Why We Optimize NEW

The trick: a binary tree's parent-child links only go one direction (down), but distance to nodes that require going *up* through ancestors needs a way to traverse upward too — so first do a traversal building a **parent-pointer map** (recording each node's parent, since the tree itself doesn't store this), then treat the tree as an undirected graph (where you can move to a parent, left child, or right child, all as equally valid 'neighbours') and run a **BFS starting from the target node**, stopping exactly K levels out. This converts a tree-distance problem into a graph BFS problem by adding the missing 'upward' links via the parent map — the same core BFS technique from Level Order, just applied to a graph-like view of the tree rather than the tree's native root-to-leaf direction, and directly foreshadowing BFS's central role in the upcoming Graphs chapter.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find all nodes at distance K from a given node.

Visual Diagram

```
Tree:
                  1
              /   \
           2       3
           / \     / \
```

```
          4    5 6    7

       Target: 2, K = 1
       Nodes at distance 1: [1, 4, 5]

       Target: 2, K = 2
       Nodes: [3, 4, 5, 6, 7? Wait, 6 and 7 are distance 3 from 2]

       Distance 2 from 2:
       - Through parent: 2→1→3 → 3
       - Through left child: 2→4 (distance 1) → no child
       - Through right child: 2→5 (distance 1) → no child
       Actually 4 and 5 are distance 1, not 2

       Distance 2 from 2: [3, 6, 7]
```

Solution Approach

Concept: Find target, then search in subtrees.

Algorithm

```
Algorithm NODES_AT_DISTANCE_K(root, target, k):
           map = new HashMap<Node, Node>() // parent map
           DFS to fill parent map

           queue = Queue()
           queue.add(target)
           visited = HashSet()
           visited.add(target)
           distance = 0
           result = []

           While queue not empty and distance <= k:
               size = queue.size()
               For i = 0 to size-1:
                   node = queue.poll()
                   If distance == k: result.add(node.data)
                   // Add parent, left, right if not visited
                   If node.parent not visited: add to queue
                   If node.left not visited: add to queue
                   If node.right not visited: add to queue
               distance++

           Return result
```

Java Implementation

```
class Solution {
           public List<Integer> distanceK(Node root, Node target, int k) {
```

```
            Map<Node, Node> parent = new HashMap<>();
            getParentMap(root, null, parent);

            Queue<Node> queue = new LinkedList<>();
            Set<Node> visited = new HashSet<>();
            queue.add(target);
            visited.add(target);
            List<Integer> result = new ArrayList<>();
            int distance = 0;

            while (!queue.isEmpty() && distance <= k) {
                int size = queue.size();
                for (int i = 0; i < size; i++) {
                    Node node = queue.poll();
                    if (distance == k) {
                        result.add(node.data);
                    }
                    if (node.left != null && !visited.contains(node.left)) {
                        queue.add(node.left);
                        visited.add(node.left);
                    }
                    if (node.right != null && !visited.contains(node.right)) {
                        queue.add(node.right);
                        visited.add(node.right);
                    }
                    if (parent.get(node) != null && !
visited.contains(parent.get(node))) {
                        queue.add(parent.get(node));
                        visited.add(parent.get(node));
                    }
                }
                distance++;
            }
            return result;
        }

        private void getParentMap(Node node, Node parent, Map<Node, Node> map) {
            if (node == null) return;
            map.put(node, parent);
            getParentMap(node.left, node, map);
            getParentMap(node.right, node, map);
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

274. Minimum Time to Burn Binary Tree

Understanding the Problem NEW

Given a target node that starts on fire, and fire spreads to adjacent nodes (parent, left child, right child) one time-unit per step, find the minimum time needed for the fire to reach every node in the tree.

Brute Force Approach NEW

Simulate the fire spreading minute by minute across the whole tree structure directly using the tree's native (parent-to-child only) links — but since fire can also spread *upward* to a parent, the tree's one-directional child links alone aren't sufficient to simulate this correctly without extra bookkeeping.

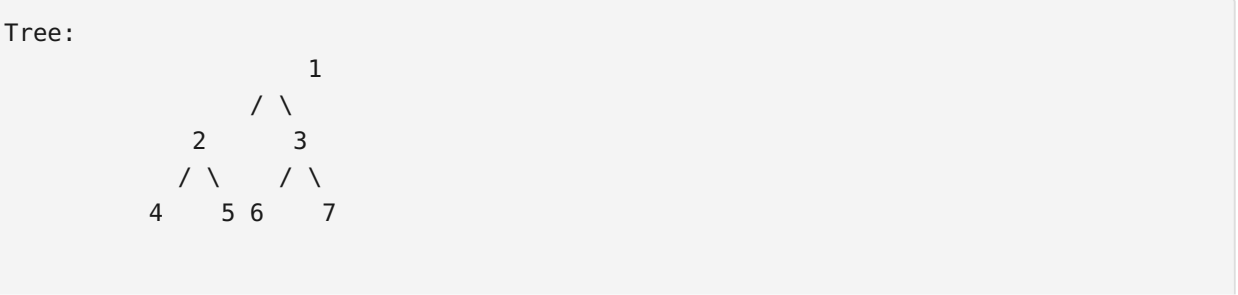
Why We Optimize NEW

Exactly the same fix as Nodes at Distance K (273): build a parent-pointer map first, then run a **BFS from the target node** treating parent/left/right as equally valid neighbours (since fire doesn't care about the 'down-only' direction trees normally impose) — the total time to burn the whole tree is simply the number of BFS levels needed to reach every node (the eccentricity of the target node in this now-undirected view of the tree). This is essentially the same technique as problem 273, extended to find the maximum distance to *any* node rather than just nodes at exactly distance K — another direct reuse of 'add parent pointers, then BFS' to handle upward movement that plain trees don't natively support.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find time to burn entire tree from a target node.

Visual Diagram



```
       Burn from 2:
       Time 0: 2
       Time 1: 1, 4, 5
       Time 2: 3, 6, 7
       Total time = 2

       Same as finding max distance from target
```

Solution Approach

Concept: BFS from target with parent map.

Algorithm

```
Algorithm BURN_TIME(root, target):
           parent = getParentMap(root)
           queue = Queue()
           queue.add(target)
           visited = HashSet()
           visited.add(target)
           time = 0

           While queue not empty:
               size = queue.size()
               burned = false
               For i = 0 to size-1:
                   node = queue.poll()
                   If node.left not visited:
                       burned = true
                       queue.add(node.left)
                       visited.add(node.left)
                   If node.right not visited:
                       burned = true
                       queue.add(node.right)
                       visited.add(node.right)
                   If parent.get(node) not visited:
                       burned = true
                       queue.add(parent.get(node))
                       visited.add(parent.get(node))
               If burned: time++

           Return time
```

Java Implementation

```
class Solution {
           public int burnTree(Node root, Node target) {
               Map<Node, Node> parent = new HashMap<>();

              getParentMap(root, null, parent);
```

```
            Queue<Node> queue = new LinkedList<>();
            Set<Node> visited = new HashSet<>();
            queue.add(target);
            visited.add(target);
            int time = 0;

            while (!queue.isEmpty()) {
                int size = queue.size();
                boolean burned = false;

                for (int i = 0; i < size; i++) {
                    Node node = queue.poll();

                    if (node.left != null && !visited.contains(node.left)) {
                        burned = true;
                        queue.add(node.left);
                        visited.add(node.left);
                    }
                    if (node.right != null && !visited.contains(node.right)) {
                        burned = true;
                        queue.add(node.right);
                        visited.add(node.right);
                    }
                    if (parent.get(node) != null && !
visited.contains(parent.get(node))) {
                        burned = true;
                        queue.add(parent.get(node));
                        visited.add(parent.get(node));
                    }
                }
                if (burned) time++;
            }
            return time;
        }

        private void getParentMap(Node node, Node parent, Map<Node, Node> map) {
            if (node == null) return;
            map.put(node, parent);
            getParentMap(node.left, node, map);
            getParentMap(node.right, node, map);
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

275. Count Total Nodes in Complete Binary Tree

Understanding the Problem NEW

Given a **Complete** Binary Tree specifically (every level full except possibly the last, which fills left to right with no gaps), count the total number of nodes faster than a plain O(n) traversal would.

Brute Force Approach NEW

Visit every single node with a standard traversal and count them — correct, O(n), but ignores the extra structural guarantee ('complete tree') the problem specifically provides.

Why We Optimize NEW

Exploit the completeness guarantee: for any node, compute the height by always going **left** only, and separately the height by always going **right** only — if these two heights are equal, the subtree rooted here is a *perfect*2^height - 1, with no need to visit every individual node; if the heights differ, recurse into both children separately and combine their counts (only one side, at most, will actually be 'incomplete' and require deeper recursion, thanks to the completeness property). This achieves O(log²n) time — recursing O(log n) levels deep, each doing O(log n) work to compute left/right height — dramatically better than the O(n) a general (non-complete) tree would require, purely by exploiting the specific structural guarantee this problem provides that general trees don't have.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count nodes in complete binary tree faster than O(n).

Visual Diagram

```
Complete Binary Tree:
                1
              /    \
            2       3
          / \     / \
        4     5 6    7
```

```
       Count = 7

       Complete binary tree property:
       If leftHeight == rightHeight: nodes = 2^height - 1
       Else: 1 + count(left) + count(right)
```

Solution Approach

Concept: Use complete tree property.

Algorithm

```
Algorithm COUNT_NODES(root):
           If root == null: Return 0
           leftHeight = getLeftHeight(root)
           rightHeight = getRightHeight(root)

           If leftHeight == rightHeight:
               Return (1 << leftHeight) - 1
           Else:
               Return 1 + COUNT_NODES(root.left) + COUNT_NODES(root.right)
```

Java Implementation

```
class Solution {
           public int countNodes(Node root) {
               if (root == null) return 0;
               int leftHeight = getLeftHeight(root);
               int rightHeight = getRightHeight(root);

                 if (leftHeight == rightHeight) {
                     return (1 << leftHeight) - 1;
                 }
                 return 1 + countNodes(root.left) + countNodes(root.right);
           }

           private int getLeftHeight(Node node) {
               int height = 0;
               while (node != null) {
                   height++;
                   node = node.left;
               }
               return height;
           }

           private int getRightHeight(Node node) {
               int height = 0;
               while (node != null) {

                   height++;
```

```
                node = node.right;
            }
            return height;
        }
    }
```

Time & Space Complexity

Time Complexity: O(log² n) | Space Complexity: O(log n)

276-278. Construct Binary Trees from Traversals

Understanding the Problem NEW

Given two of a tree's traversal sequences (like Preorder + Inorder, or Postorder + Inorder), reconstruct the original tree structure that would produce exactly those sequences.

Brute Force Approach NEW

Without a clear plan, you might try many possible tree shapes and check if replaying their traversals matches the given sequences — extremely wasteful, since there's a direct constructive method available instead of a guess-and-check search.

Why We Optimize NEW

The key insight: in **Preorder**, the very first element is always the root; in **Inorder**, everything to the *left* of that same root's position belongs to the left subtree, and everything to the *right* belongs to the right subtree. So you can recursively: take the current preorder's first element as the root, find that value's position in the current inorder slice to split it into left/right portions, and recurse to build the left and right subtrees from the corresponding slices of both sequences. Using a hashmap to look up each value's position in the inorder sequence (instead of scanning for it each time) brings this down to O(n) total, rather than an $O(n^2)$ approach that re-scans the inorder sequence for every node, or a combinatorial guess-and-check search — this exact 'first element = root, split by its position in the other sequence' technique works symmetrically for Postorder + Inorder too, just using postorder's *last* element as the root instead of preorder's first.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Construct trees from given traversals.

Visual Diagram

```
Inorder: [4, 2, 5, 1, 6, 3, 7]
       Preorder: [1, 2, 4, 5, 3, 6, 7]
```

```
      Tree:
                  1
               /  \
            2       3
            / \    / \
          4    5 6    7

      Build process:
      Preorder[0] = 1 → root
      Find 1 in inorder → index 3
      Left inorder: [4,2,5], Right inorder: [6,3,7]
      Left preorder: [2,4,5], Right preorder: [3,6,7]
      Recursively build
```

Java Implementation

```
class Solution {
           // Build from inorder and preorder
           public Node buildTreeInPre(int[] inorder, int[] preorder) {
               Map<Integer, Integer> map = new HashMap<>();
               for (int i = 0; i < inorder.length; i++) {
                   map.put(inorder[i], i);
               }
               return buildInPre(preorder, 0, preorder.length - 1,
                                 inorder, 0, inorder.length - 1, map);
           }

           private Node buildInPre(int[] preorder, int preStart, int preEnd,
                                   int[] inorder, int inStart, int inEnd,
                                   Map<Integer, Integer> map) {
               if (preStart > preEnd || inStart > inEnd) return null;

                    int rootVal = preorder[preStart];
                    Node root = new Node(rootVal);
                    int rootIndex = map.get(rootVal);
                    int leftSize = rootIndex - inStart;

                    root.left = buildInPre(preorder, preStart + 1, preStart +
leftSize,
                                           inorder, inStart, rootIndex - 1, map);
                    root.right = buildInPre(preorder, preStart + leftSize + 1,
preEnd,
                                            inorder, rootIndex + 1, inEnd, map);
                    return root;
           }

           // Build from inorder and postorder
           public Node buildTreeInPost(int[] inorder, int[] postorder) {
               Map<Integer, Integer> map = new HashMap<>();
               for (int i = 0; i < inorder.length; i++) {
                   map.put(inorder[i], i);
```

```
            }
            return buildInPost(postorder, 0, postorder.length - 1,
                               inorder, 0, inorder.length - 1, map);
        }

        private Node buildInPost(int[] postorder, int postStart, int postEnd,
                                 int[] inorder, int inStart, int inEnd,
                                 Map<Integer, Integer> map) {
            if (postStart > postEnd || inStart > inEnd) return null;

            int rootVal = postorder[postEnd];
            Node root = new Node(rootVal);
            int rootIndex = map.get(rootVal);
            int leftSize = rootIndex - inStart;

            root.left = buildInPost(postorder, postStart, postStart + leftSize -
1,
                                    inorder, inStart, rootIndex - 1, map);
            root.right = buildInPost(postorder, postStart + leftSize, postEnd -
1,
                                     inorder, rootIndex + 1, inEnd, map);
            return root;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

279. Serialize and Deserialize Binary Tree

Understanding the Problem NEW

Design a way to convert a binary tree into a single string (serialize), and then reconstruct the exact original tree from that string (deserialize) — useful for saving a tree to disk or sending it over a network.

Brute Force Approach NEW

You might try to serialize using just one traversal's sequence (e.g. only preorder values), but a single traversal sequence alone is generally *not enough* to uniquely reconstruct a tree's shape (multiple different trees can share the same preorder sequence, for instance), so this naive approach can fail to reconstruct correctly.

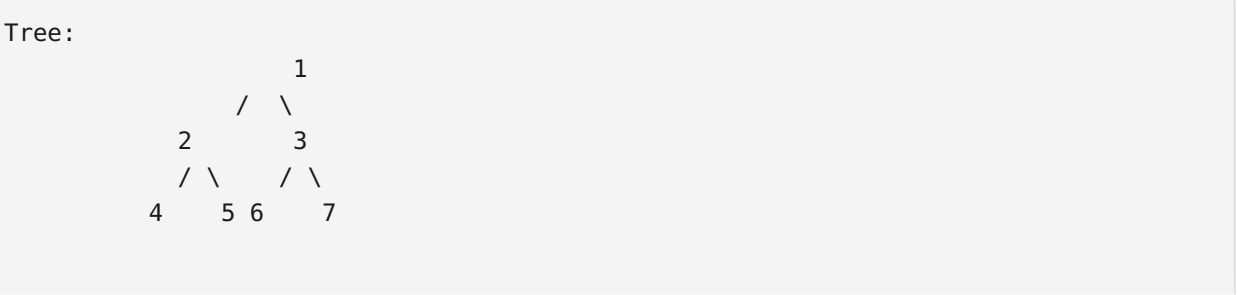
Why We Optimize NEW

The fix: serialize using preorder traversal, but explicitly include **markers for null/missing children** (e.g. writing 'N' whenever a child is absent, not just skipping it silently) — this extra information about *where* the gaps are is exactly what disambiguates the tree's shape, making a single traversal sufficient after all. Deserializing then just replays the same sequence with a pointer/index, recursively rebuilding left and right children (creating a null node whenever the 'N' marker is encountered), mirroring exactly how the tree was written out. Both serialize and deserialize run in O(n) time — the key lesson being that a traversal alone is ambiguous, but a traversal *plus explicit null markers* is not.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Convert tree to string and back.

Visual Diagram



```
       Serialized: [1,2,3,4,5,6,7] (level order)

       Using preorder with null markers:
       Serialized: [1,2,4,null,null,5,null,null,3,6,null,null,7,null,null]

       Deserialize:
       Build from preorder sequence
```

Solution Approach

Concept: Preorder traversal with null markers.

Algorithm

```
Algorithm SERIALIZE(root):
           result = []
           serializeHelper(root, result)
           Return String.join(",", result)

       Algorithm serializeHelper(node, result):
           If node == null:
               result.add("null")
               Return
           result.add(String.valueOf(node.data))
           serializeHelper(node.left, result)
           serializeHelper(node.right, result)

       Algorithm DESERIALIZE(data):
           queue = Queue of data split by comma
           Return deserializeHelper(queue)

       Algorithm deserializeHelper(queue):
           val = queue.poll()
           If val == "null": Return null
           node = new Node(Integer(val))
           node.left = deserializeHelper(queue)
           node.right = deserializeHelper(queue)
           Return node
```

Java Implementation

```
class Solution {
           // Serialize
           public String serialize(Node root) {
               StringBuilder sb = new StringBuilder();
               serializeHelper(root, sb);
               return sb.toString();
           }

          private void serializeHelper(Node node, StringBuilder sb) {
```

```
            if (node == null) {
                sb.append("null,");
                return;
            }
            sb.append(node.data).append(",");
            serializeHelper(node.left, sb);
            serializeHelper(node.right, sb);
        }

        // Deserialize
        public Node deserialize(String data) {
            Queue<String> queue = new
LinkedList<>(Arrays.asList(data.split(",")));
            return deserializeHelper(queue);
        }

        private Node deserializeHelper(Queue<String> queue) {
            String val = queue.poll();
            if (val.equals("null")) return null;
            Node node = new Node(Integer.parseInt(val));
            node.left = deserializeHelper(queue);
            node.right = deserializeHelper(queue);
            return node;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

280-281. Morris Traversals

Understanding the Problem NEW

Perform tree traversals (Preorder, Inorder) using O(1) extra space — no recursion (which uses O(h) call-stack space) and no explicit stack (which uses O(h) space too).

Brute Force Approach NEW

Ordinary recursive or stack-based traversals (as in problems 248-249, 252-255) both use O(h) extra space for tracking 'where to return to' — perfectly fine for most trees, but this problem specifically challenges you to eliminate that space entirely.

Why We Optimize NEW

Morris Traversal achieves this with a clever trick: instead of using a stack to remember 'where to come back to' after finishing a left subtree, temporarily create a **threaded link** — find the current node's in-order predecessor (the rightmost node in its left subtree) and set *that* node's normally-null right pointer to point back to the current node. This lets you navigate back up the tree later without any stack, since the tree's own (temporarily borrowed) pointers now encode the return path. Once you've used a threaded link to return to a node, you detect and remove it (restoring the tree to its original structure) before moving on. This achieves true O(1) extra space (beyond the traversal output itself) at the cost of temporarily modifying and then carefully restoring the tree's pointers — a genuinely different category of technique from anything else in this chapter, trading a bit of pointer-manipulation complexity for a real space-complexity improvement from O(h) to O(1).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Traverse tree without recursion or stack (O(1) space).

Visual Diagram

```
Morris Inorder:
                1
```

```
               /    \
             2       3
            / \     / \
          4     5 6    7

       Algorithm: Create threaded links to predecessor
       1. If left child exists, find predecessor (rightmost in left subtree)
       2. Set predecessor.right = current (create thread)
       3. Move left
       4. If no left child, visit node, move right
```

Java Implementation

```
class Solution {
           // Morris Inorder Traversal
           public List<Integer> morrisInorder(Node root) {
               List<Integer> result = new ArrayList<>();
               Node current = root;

                while (current != null) {
                    if (current.left == null) {
                        result.add(current.data);
                        current = current.right;
                    } else {
                        // Find predecessor
                        Node predecessor = current.left;
                        while (predecessor.right != null && predecessor.right !=
current) {
                             predecessor = predecessor.right;
                        }

                           if (predecessor.right == null) {
                               // Create thread
                               predecessor.right = current;
                               current = current.left;
                           } else {
                               // Remove thread and visit
                               predecessor.right = null;
                               result.add(current.data);
                               current = current.right;
                           }
                    }
                }
                return result;
           }

           // Morris Preorder Traversal
           public List<Integer> morrisPreorder(Node root) {
               List<Integer> result = new ArrayList<>();
               Node current = root;

                while (current != null) {
```

```
                    if (current.left == null) {
                        result.add(current.data);
                        current = current.right;

                } else {
                    Node predecessor = current.left;
                    while (predecessor.right != null && predecessor.right !=
current) {
                         predecessor = predecessor.right;
                    }

                    if (predecessor.right == null) {
                        result.add(current.data);
                        predecessor.right = current;
                        current = current.left;
                    } else {
                        predecessor.right = null;
                        current = current.right;
                    }
                }
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

282. Flatten Binary Tree to LinkedList

Understanding the Problem NEW

Rearrange a binary tree in-place into a 'linked list' shape — every node's left child becomes null, and its right child becomes the next node in preorder traversal order (so following right pointers alone reconstructs the exact preorder sequence).

Brute Force Approach NEW

You could first compute the full preorder traversal into a separate list (O(n) extra space), then rebuild the tree's pointers to match that list's order — correct, but uses O(n) extra space for the intermediate list when the rearrangement could be done without it.

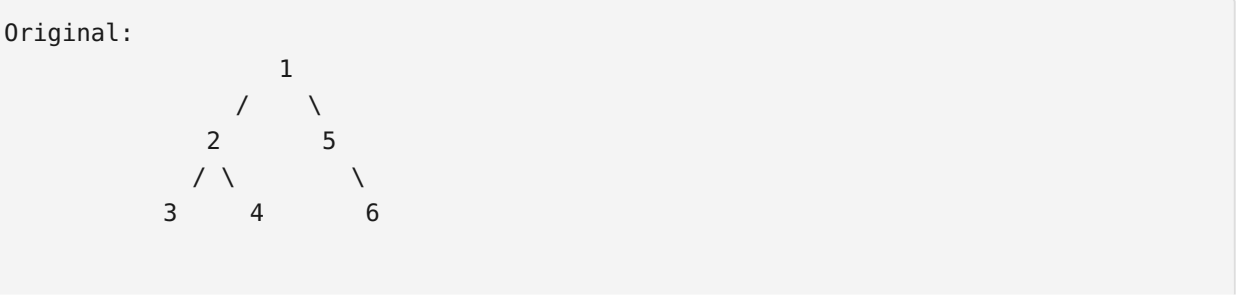
Why We Optimize NEW

A clean in-place approach reuses the exact same trick as Morris Traversal (280-281): for each node, if it has a left child, find the **rightmost node in that left subtree** (the predecessor) and attach the current node's original right subtree there — then move the entire left subtree over to become the new right subtree, and clear the left pointer. Repeating this at every node (moving to the new right child each time) flattens the tree in a single O(n) pass using O(1) extra space, by directly rewiring existing pointers in a Morris-traversal-like manner rather than building an intermediate list and reconstructing from it.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Flatten tree to linked list in preorder.

Visual Diagram



```
        Flattened (right child as next):
        1 → 2 → 3 → 4 → 5 → 6

        Morris-like traversal:
        For each node:
          If left exists:
            Find rightmost of left subtree
            Connect rightmost.right = node.right
            node.right = node.left
            node.left = null
          Move to right
```

Solution Approach

Concept: In-place Morris-like flattening.

Algorithm

```
Algorithm FLATTEN(root):
            current = root

             While current != null:
                 If current.left != null:
                     // Find rightmost of left subtree
                     rightmost = current.left
                     While rightmost.right != null:
                         rightmost = rightmost.right

                       // Connect rightmost to current.right
                       rightmost.right = current.right
                       current.right = current.left
                       current.left = null

                  current = current.right
```

Java Implementation

```
class Solution {
            public void flatten(Node root) {
                Node current = root;

                  while (current != null) {
                      if (current.left != null) {
                          // Find rightmost of left subtree
                          Node rightmost = current.left;
                          while (rightmost.right != null) {
                              rightmost = rightmost.right;
                          }

                      // Connect rightmost to current.right
```

```
                    rightmost.right = current.right;
                    current.right = current.left;
                    current.left = null;
                }
                current = current.right;
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

Binary Tree Summary

Problem	Difficulty	Time	Space	Key Technique
Traversals	Basic	O(n)	O(h)	DFS/BFS
Height	Medium	O(n)	O(h)	Recursion
Balanced	Medium	O(n)	O(h)	Recursive check
Diameter	Medium	O(n)	O(h)	Height + path
Max Path Sum	Medium	O(n)	O(h)	Return max gain
Identical	Medium	O(n)	O(h)	Recursive comparison
Zigzag	Medium	O(n)	O(n)	Queue + toggle
Boundary	Medium	O(n)	O(h)	Three parts
Vertical	Medium	O(n log n)	O(n)	HD map
Top View	Medium	O(n)	O(n)	BFS + first
Bottom View	Medium	O(n)	O(n)	BFS + last
Left/Right View	Medium	O(n)	O(n)	Level order
Symmetric	Medium	O(n)	O(h)	Mirror check
Root to Path	Hard	O(n)	O(h)	Backtracking
LCA	Hard	O(n)	O(h)	Recursive search
Max Width	Hard	O(n)	O(n)	Indexing
Children Sum	Hard	O(n)	O(h)	Recursive check
Distance K	Hard	O(n)	O(n)	Parent map + BFS
Burn Time	Hard	O(n)	O(n)	Parent map + BFS
Count Complete	Hard	$O(\log^2 n)$	O(log n)	Height property
Build Tree	Hard	O(n)	O(n)	Mapping indexes
Serialize	Hard	O(n)	O(n)	Preorder with null
Morris Traversal	Hard	O(n)	O(1)	Threaded links
Flatten	Hard	O(n)	O(1)	Morris-like